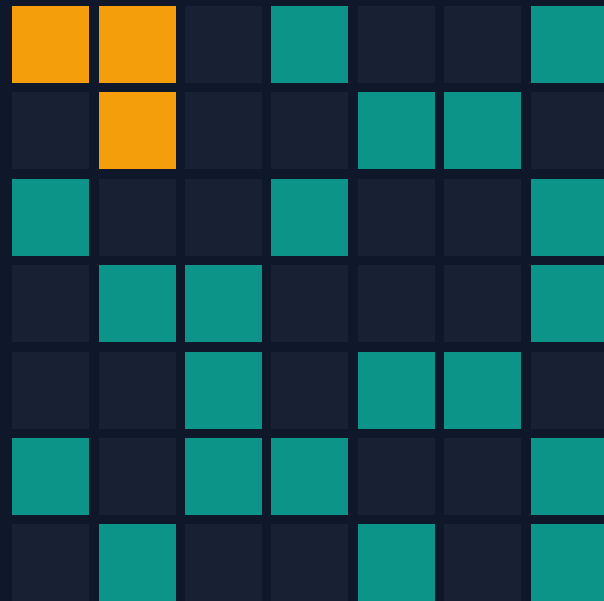


Contagem paralela de objetos em uma matriz binária

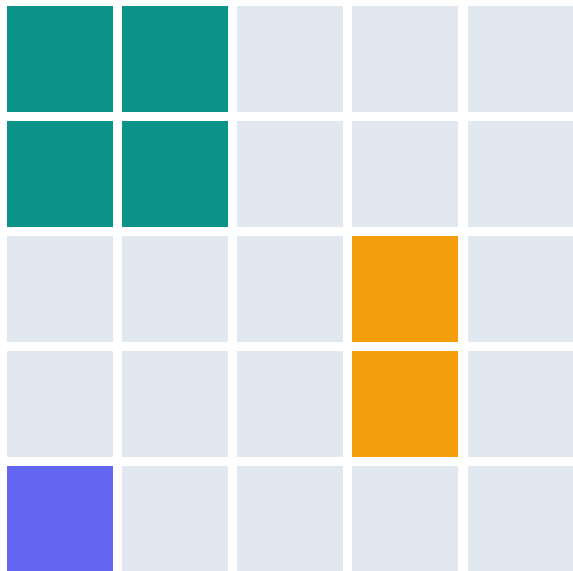
Sequencial × paralela com Pthreads

Louise e Luca · Sistemas Operacionais 2026/II
PUCRS · Prof. Filipo Mór

Comparar uma implementação sequencial com uma paralela usando Pthreads.

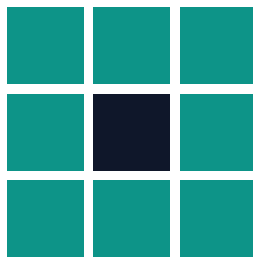


O problema: o que é um objeto?



0 = fundo · 1 = parte de um objeto · 3 objetos

Conectividade 8

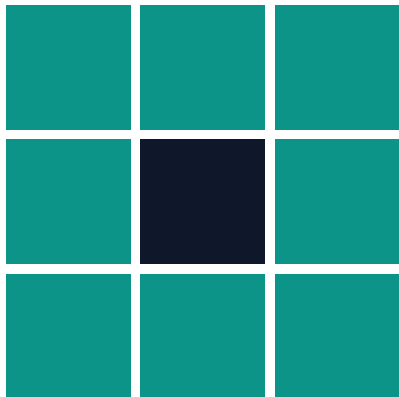


Lados e diagonais: cada célula tem até oito vizinhos.

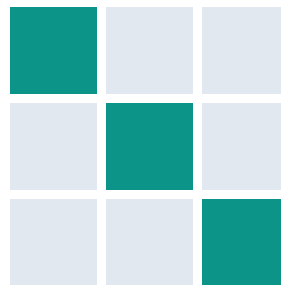
A pergunta

Quantos objetos existem na matriz? No exemplo ao lado, três.

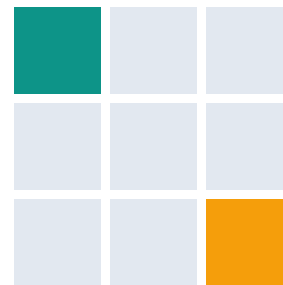
Conectividade 8: quem é vizinho?



Cada célula tem até 8 vizinhos: 4 pelos lados e 4 pelas diagonais.



Mesmo objeto: ligados na diagonal



Objetos diferentes: sem contato

Sem a diagonal a contagem mudaria. Por isso a conectividade 8 precisa valer também nas fronteiras entre faixas.

O desafio do trabalho

1

Referência sequencial

Um único fluxo de controle:
base de correção e de
comparação de tempo.

2

Versão paralela

Pelo menos duas threads
dividindo o cálculo, em
quantidade configurável.

3

Mesmo resultado

Conectividade 8 preservada,
sem contagem duplicada nem
condição de corrida.

Um objeto pode ocupar duas regiões: somar as contagens locais não basta.

Solução sequencial (a referência)

1

Percorrer a matriz

2

Encontrar um 1
ainda não visitado

3

Incrementar a
contagem

4

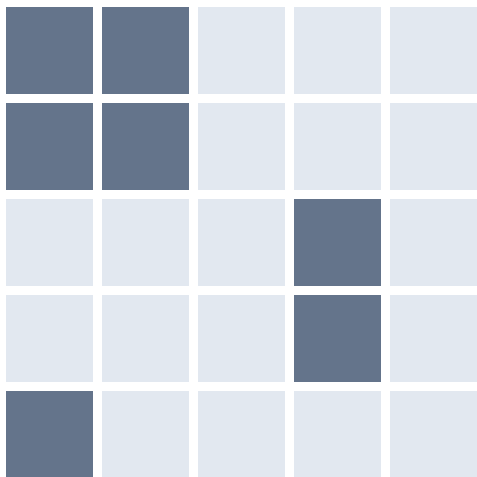
Marcar todo o
objeto com flood
fill

Pilha explícita no heap, sem recursão

Evita chamadas recursivas profundas em objetos grandes. Esta versão é a referência para validar a paralela.

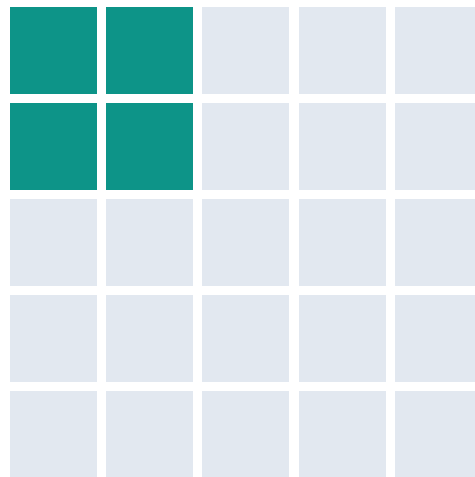
Duas matrizes: problema e anotação

matriz (nunca muda)



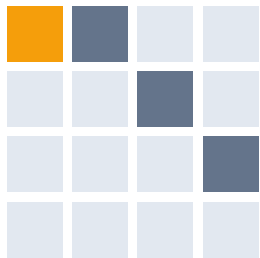
Ao achar o 1 em (0,0), o flood fill marca o objeto inteiro

visitado (começa toda 0)

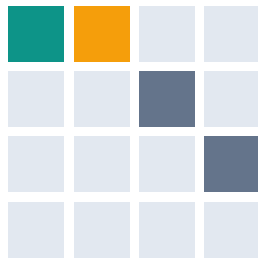


Objeto novo: $\text{matriz} = 1$ e $\text{visitado} = 0 \rightarrow \text{total} + 1$

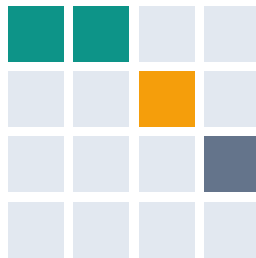
Flood fill passo a passo



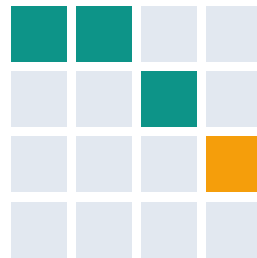
1. Marca e empilha (0,0)



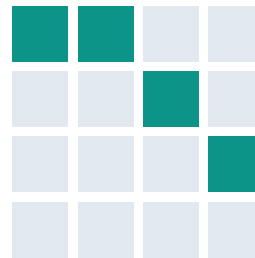
2. Tira (0,0); empilha (0,1)



3. Tira (0,1); empilha (1,2)



4. Tira (1,2); empilha (2,3)



5. Tira (2,3); pilha vazia

Cinza: terra ainda não visitada · Âmbar: na pilha · Verde: já marcada

Cada célula retirada olha seus 8 vizinhos e empilha os 1 ainda não marcados.

Por que uma pilha explícita?



(1,2) topo

(1,1)

(1,0)

Os 3 vizinhos novos entram na pilha de uma vez e saem um por vez.

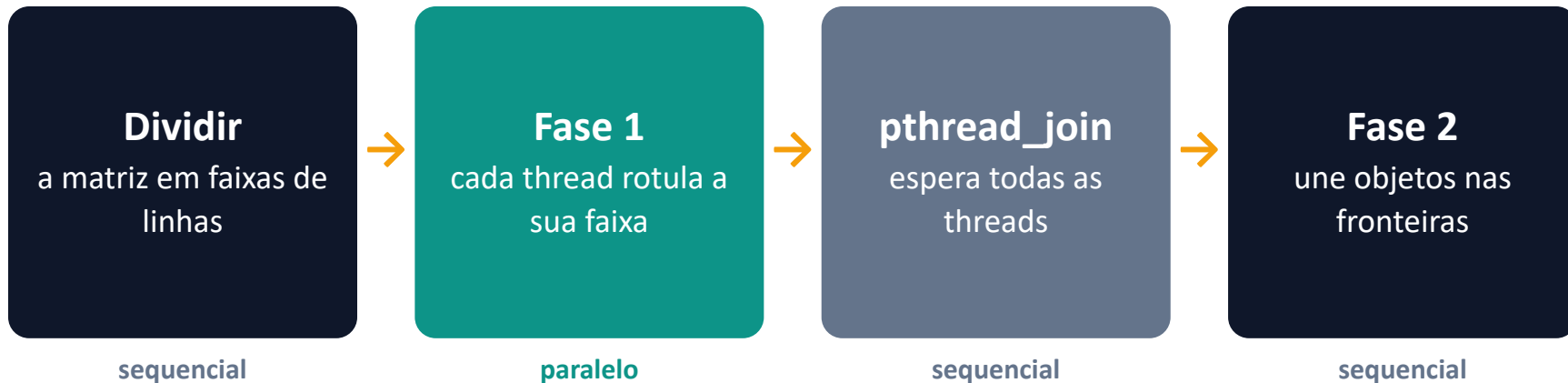
Recursão

Cada chamada ocupa a pilha de chamadas do processo, que é pequena. Um objeto com milhões de células a estoura.

Pilha explícita (heap)

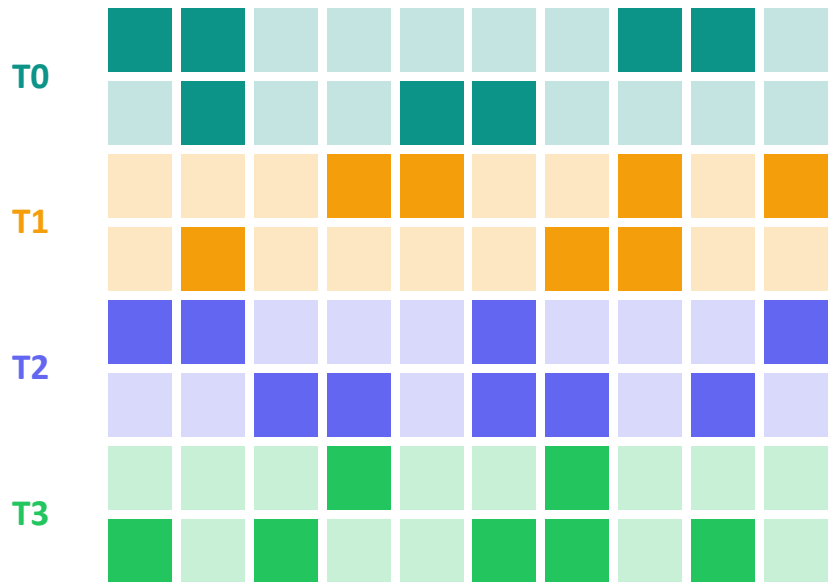
A célula é marcada ao entrar, então entra no máximo uma vez: o vetor de linhas $\times$ colunas nunca enche.

Estratégia: duas fases



Total = soma das contagens locais – uniões efetivas

Divisão da matriz entre threads



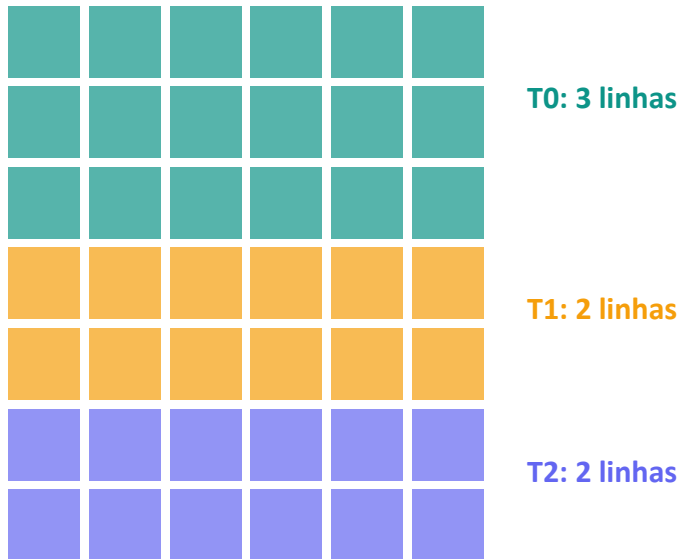
- Uma faixa contígua de linhas por thread
- Tamanhos diferem em no máximo uma linha
- Número de threads configurável (no máximo uma por linha)

1 • Identificação
local (paralela)



2 • Consolidação
(thread principal)

Quando a divisão não é exata



7 linhas e 3 threads

$$\text{base} = 7 \div 3 = 2$$

$$\text{resto} = 7 \bmod 3 = 1$$

As primeiras 'resto' faixas ganham 1 linha extra:
3, 2 e 2.

Mais threads que linhas? Uma thread por linha.

Fase 1: identificação dos componentes

Faixa da thread 0 (linhas 0 a 2)

						7	7
	10	10	10	10		7	
		10	10				

Faixa da thread 1 (linhas 3 a 5)

			28	28			
				28			40
41	41					40	40

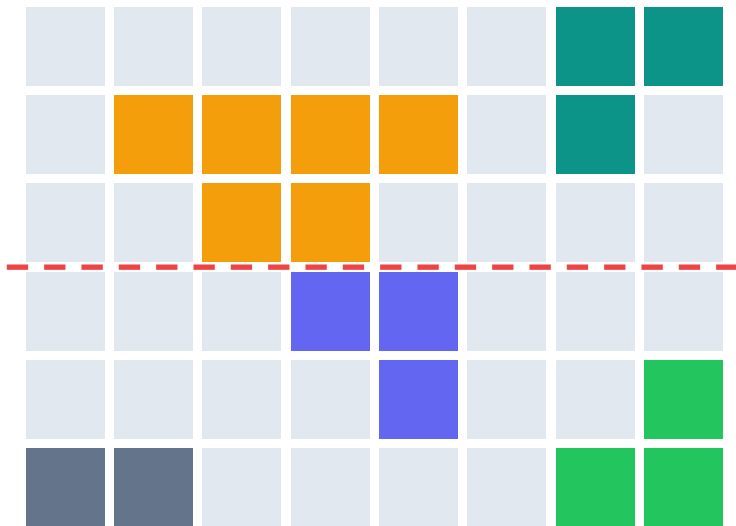
rótulo = linha × colunas + coluna + 1

É a posição da primeira célula do objeto local.

Único na matriz inteira: threads diferentes nunca repetem rótulo.

Exemplo: o objeto B começa em (1,1), então $1 \times 8 + 1 + 1 = 10$.

Fase 1: o flood fill não sai da faixa



- Cada thread só olha vizinhos com $ini \leq linha < fim$
- A parte do objeto na faixa de baixo é trabalho da outra thread
- Resultado: o objeto central vira 2 componentes locais (rótulos 10 e 28)

```
if (nl < ini || nl >= fim || ...)
    continue;
```

Compartilhamento e sincronização

Dado	Como é acessado
Matriz	Somente leitura
Rótulos	Escrita apenas na faixa da thread
Union-find (pai)	Cada thread escreve só nos próprios rótulos
Pilha	Uma por thread
Resultado local	Lido após pthread_join

Paralelo: fase 1, uma faixa por thread. Sequencial: joins e consolidação (thread principal). Sem mutexes.

O problema: objeto cortado ao meio



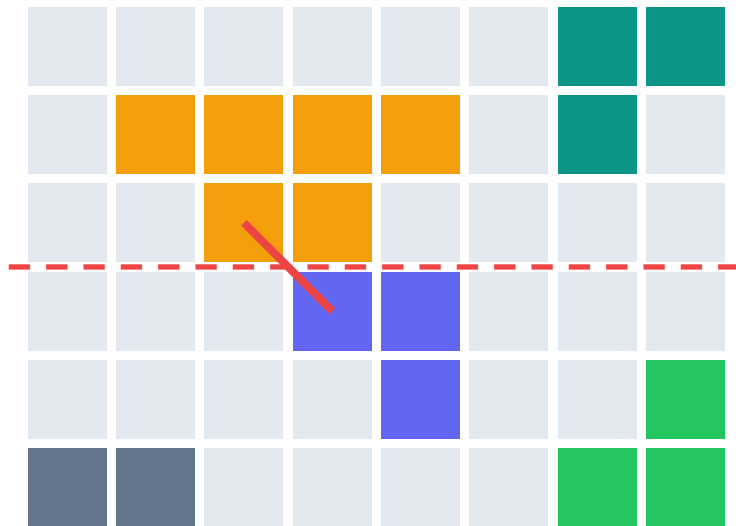
Thread 0: 1 componente

Thread 1: 1 componente

Soma: 2, mas existe 1 objeto

É preciso reconhecer que os dois componentes são o mesmo objeto.

Como unir objetos que cruzam fronteiras



Faixa superior: 2 componentes locais



vizinhos abaixo

Faixa inferior: 3 componentes locais

Soma inicial: 5 · uma união efetiva

Total = soma das contagens locais – uniões efetivas

Aqui: $5 - 1 = 4$ objetos finais

Union-find: o registro de grupos

Rótulo	pai (início)	pai (após união)
7	7	7
10	10	10
28	28	10
40	40	40
41	41	41

uniao(10, 28)

Raízes diferentes: $\text{pai}[28] = 10$, retorna 1 e o total desce 1.

uniao(10, 28) de novo (outro contato)

Mesma raiz: retorna 0 e nada é descontado.

Total: 5 componentes locais – 1 união efetiva = 4 objetos

Demonstração do programa



```
$ make  
$ ./build/sequencial 2  
$ ./build/paralelo 2 2  
$ ./build/paralelo 2 4  
$ ./build/paralelo -f tests/adicionais/em_u.txt 4
```

make Compilação

=

Sequencial ×
paralela



Mesmo resultado
com 2 e 4 threads

Como verificamos a correção

5

casos obrigatórios

3, 4, 5, 6 e 7 objetos

7

casos adicionais

além dos obrigatórios

5

contagens de threads

1, 2, 3, 4 e 8

0

erros nos sanitizadores

segundo as verificações
registradas

Casos de destaque

xadrez

diagonais

U

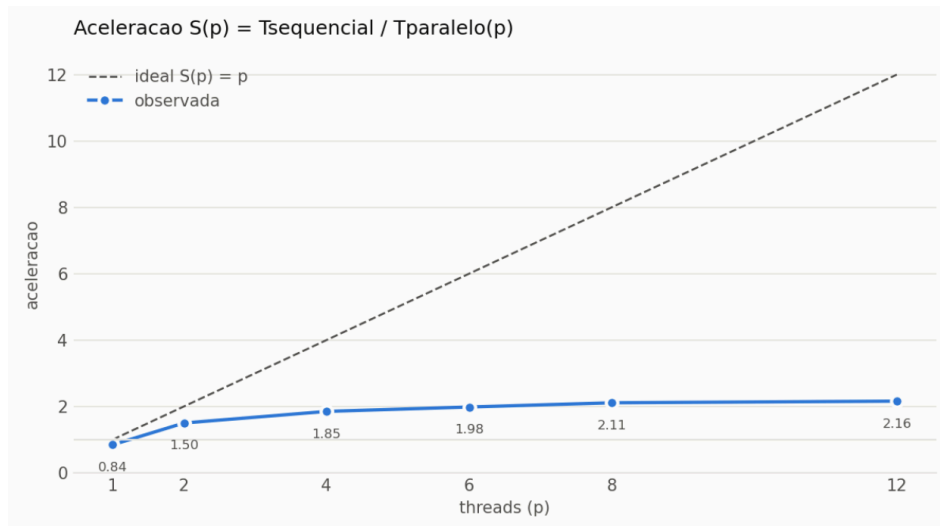
linha única

A matriz grande teve a contagem conferida por uma implementação independente em Python.

Como medimos o desempenho

- 1 Mesma matriz (2000×2000, oito objetos) em todas as configurações
- 2 Um aquecimento descartado e mediana de dez execuções
- 3 Tempo da contagem: inclui criar e esperar as threads, sem ler o arquivo
- 4 1, 2, 4, 6, 8 e 12 threads (CPU com 6 núcleos físicos e 12 lógicos)
- 5 Aceleração $S = T_{\text{sequencial}} / T_{\text{paralelo}}$ · Eficiência $E = S / \text{threads}$

Aceleração observada



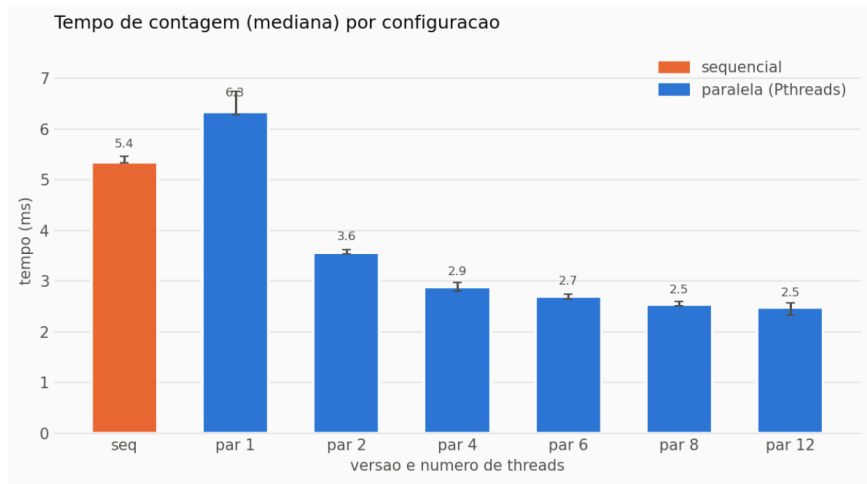
- Matriz 2000×2000, oito objetos
- Xeon: seis núcleos físicos e 12 processadores lógicos
- Um aquecimento descartado; mediana de dez execuções

Versão	Threads	Mediana	Aceleração
Sequencial	1	≈ 5,3 ms	1,00×
Paralela	1	≈ 6,3 ms	0,84×
Paralela	4	≈ 2,9 ms	1,85×
Paralela	12	≈ 2,5 ms	2,16×

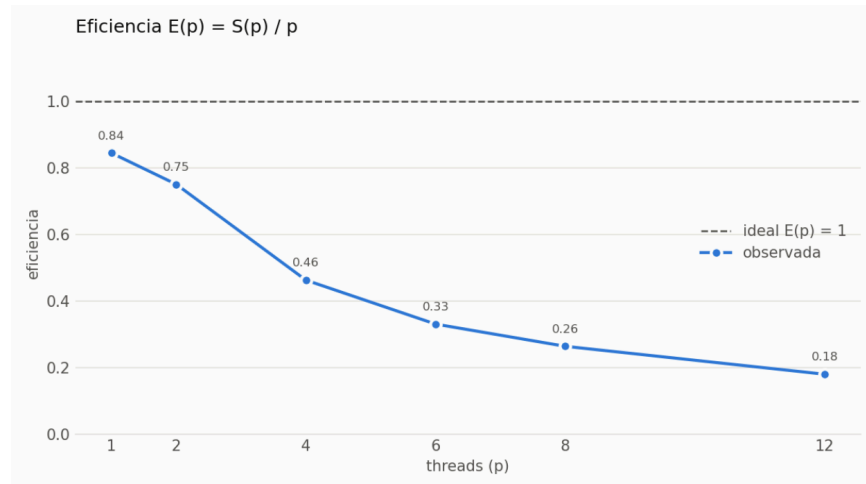
Eficiência com 12 threads: 0,18

- Tempo da contagem, sem leitura do arquivo
- $S = T_{\text{sequencial}} / T_{\text{paralelo}}; E = S / \text{threads}$

Tempo e eficiência



Tempo (mediana) por configuração



Eficiência $E(p) = S(p) / p$

A eficiência cai de 0,84 (1 thread) para 0,18 (12 threads).

Por que 12 threads não dão 12×

Sobrecarga

Criar e esperar threads; rótulos e pai usam mais memória que o visitado.

Trabalho desigual

As faixas têm o mesmo número de linhas, mas não de células de objeto.

Memória

O acesso à memória pode limitar o ganho.

Núcleos

Só 6 físicos (12 lógicos) e a fase 2 é sequencial.

Conclusões e melhorias

- ✓ Resultados corretos nos casos testados
- ✓ Consolidação preserva conexões diagonais
- ✓ Sincronização simples, por `pthread_join`

2,16×

melhor aceleração medida (12 threads)

Melhorias propostas

Distribuição dinâmica de faixas menores e redução da memória dos rótulos.